

Kafka, Redis, RabbitMQ

সিনিয়র লেভেল রিভিশন — ১৫ মিনিট। মূল ফোকাস: কোন টুল কোন সমস্যার জন্য, আর প্রোডাকশনে কী কী বিপদ হয়।

১. Kafka vs RabbitMQ vs Redis — কোনটা কখন বেছে নেবেন?

Description: তিনটাই "message পাঠায়", কিন্তু সম্পূর্ণ ভিন্ন দর্শনে। এই তুলনাটা পরিস্কার বলতে পারাই এই টপিকের সবচেয়ে সম্ভাব্য প্রশ্ন।

৬টা Key Point:

1. RabbitMQ = message broker: task queue, routing, per-message ack — "এই কাজটা কেউ একজন করো"
2. Kafka = distributed log: event stream, replay, একাধিক consumer group স্বাধীনভাবে পড়ে — "এটা ঘটেছে, যার দরকার শোনো"
3. Redis = in-memory data store, যার মধ্যে queue/pub-sub-ও করা যায় — সরলতা চাইলে; delivery guarantee দুর্বলতম
4. Message retention পার্থক্য: RabbitMQ consume হলে মুছে যায়; Kafka retention period পর্যন্ত থাকে — replay/নতুন consumer সম্ভব
5. Throughput: Kafka লাখো msg/sec (sequential disk write); RabbitMQ হাজারো — routing জটিলতায় শক্তিশালী
6. প্র্যাকটিক্যাল রুল: Laravel app-এর job queue → Redis; জটিল routing/RPC/delayed retry → RabbitMQ; event sourcing/analytics pipeline/একাধিক সিস্টেমে fan-out → Kafka

২. Redis-এর Data Structure ও বাস্তব ব্যবহার

Description: Redis শুধু key-value cache না — প্রতিটা data structure-এর একটা বাস্তব use case বলতে পারলে অভিজ্ঞতা প্রমাণ হয়।

৬টা Key Point:

1. String: cache, counter (**INCR** — atomic, তাই rate limit/view count), session, distributed flag
2. Hash: object-এর field আলাদা করে রাখা/আপডেট — পুরো JSON serialize-deserialize না করে
3. List: **LPUSH/BRPOP** — সরল queue; Laravel-এর Redis queue driver ভেতরে এটাই
4. Sorted Set (ZSET): leaderboard, rate limiter (sliding window), delayed job scheduling (score = timestamp)
5. Set: unique tracking (আজ কারা active), tag intersection; HyperLogLog — approximate unique count মেমোরি প্রায় না খেয়ে
6. TTL সব cache key-তে বাধ্যতামূলক + eviction policy জানা (**allkeys-lru** cache-এর জন্য) — নাহলে মেমোরি ভরে OOM

৩. Cache Strategy ও Invalidation

Description: "Cache invalidation is one of the two hard problems" — কোন প্যাটার্নে cache করবেন আর stale data কীভাবে সামলাবেন।

৬টা Key Point:

1. Cache-aside (সবচেয়ে কমন): আগে cache দেখো → miss হলে DB → cache-এ রেখে return; Laravel `Cache::remember` এটাই
2. Invalidation-এর দুই পথ: TTL (সহজ, একটু stale মেনে নেওয়া) vs explicit delete on write (fresh, কিন্তু সব write path মনে রাখতে হয়)
3. Key design-ই আসল: scope key-তে থাকবে (`report:{tenant}:{territory}:{month}`) — নাহলে ভুল ডেটা/অন্য tenant-এর ডেটা serve
4. Cache stampede: জনপ্রিয় key expire হলে শত request একসাথে DB-তে — সমাধান: lock করে একজন rebuild, বাকিরা অপেক্ষা/পুরনোটা খায়
5. Thundering herd-এর আরেক দাওয়াই: TTL-এ jitter (সবাই একই মুহুর্তে expire না হয়)
6. কী cache করবেন না: টাকা/স্টকের মতো মুহুর্তে-সঠিক-লাগা ডেটা আর ঘনঘন বদলানো অথচ কম পড়া ডেটা — cache লাভের অঙ্ক read:write ratio-তে

8. Message Queue-তে Delivery Guarantee ও Idempotency

Description: At-most-once, at-least-once, exactly-once — বাস্তবে কোনটা পাওয়া যায় আর duplicate message-এর সাথে কীভাবে বাঁচবেন।

৬টা Key Point:

1. At-least-once-ই বাস্তব স্ট্যান্ডার্ড: message হারাতে না, কিন্তু duplicate আসতে পারে — ack টাইমআউট/redelivery-র কারণে
2. তাই consumer idempotent বাধ্যতামূলক: processed message ID টেবিল/Redis-এ রেখে চেক, বা natural idempotent operation (upsert)
3. Exactly-once আসলে "at-least-once + idempotent processing" — এই সং উত্তরটাই সিনিয়র উত্তর
4. Ack discipline: কাজ শেষ হওয়ার পরে ack (late ack) — আগে ack দিলে crash-এ message হারায়
5. Producer পাশেও বিপদ: DB commit হলো কিন্তু publish fail (বা উল্টো) — outbox pattern: message DB-তে একই transaction-এ, আলাদা relay পাঠায়
6. Ordering: RabbitMQ single queue-এ মোটামুটি ordered, একাধিক consumer-এ ভেঙে যায়; Kafka partition-এর মধ্যে গ্যারান্টিড — order দরকার হলে একই key একই partition-এ

৫. Kafka Architecture: Topic, Partition, Consumer Group

Description: Kafka-র scale-এর রহস্য partition-এ। এই মডেলটা পরিষ্কার বললে Kafka-র বাকি সব প্রশ্ন সহজ হয়ে যায়।

৬টা Key Point:

1. Topic = logical stream; ভেতরে N partition — প্রতিটা partition একটা ordered, append-only log
2. Producer key দিয়ে partition ঠিক হয় (hash) — একই order_id সবসময় একই partition-এ, তাই সেই key-র order রক্ষা
3. Consumer group: প্রতিটা partition group-এর এক consumer-ই পড়ে — partition সংখ্যাই parallelism-এর সিলিং
4. ভিন্ন consumer group স্বাধীন — billing group আর analytics group একই stream আলাদা offset-এ পড়ে; এটাই fan-out শক্তি
5. Offset consumer-এর দায়িত্বে — commit করা offset থেকে resume; replay মানে offset পিছিয়ে আবার পড়া
6. Rebalancing: consumer যোগ/বিয়োগে partition পুনর্বণ্টন — সাময়িক pause হয়; ঘনঘন rebalance (flaky consumer) প্রোডাকশনের কমন ব্যথা

৬. RabbitMQ Exchange, Routing ও Dead Letter Queue

Description: RabbitMQ-র শক্তি routing-এ। Exchange type আর DLQ দিয়ে ব্যর্থ message সামলানো — প্রোডাকশন queue ডিজাইনের মূল প্রশ্ন।

৬টা Key Point:

1. Producer exchange-এ পাঠায়, exchange binding rule অনুযায়ী queue-তে দেয় — producer queue-র নামও জানে না (decoupling)
2. Exchange types: direct (exact key match), topic (`order.*` wildcard), fanout (সব queue-তে broadcast), headers (বিরল)
3. Prefetch (QoS) সেট করা: worker একসাথে কতগুলো unacked নেবে — না দিলে এক worker সব গিলে বাকিরা বসে থাকে
4. Dead Letter Exchange (DLX): reject/TTL-expired message আলাদা queue-তে — poison message মূল queue আটকায় না, পরে মানুষ দেখে
5. Retry with backoff-এর কৌশল: DLX + per-queue TTL দিয়ে delayed retry loop — সাথে retry count header, সীমা পেরোলে final DLQ
6. Durability trio মনে রাখা: durable queue + persistent message + publisher confirm — তিনটাই থাকলে তবে broker restart-এ message বাঁচে

৭. Redis Persistence, HA ও Cluster

Description: "Redis restart হলে ডেটা যায়?" — persistence অপশন আর high availability সেটআপ; cache-এর বাইরে Redis-কে ভরসা করার শর্ত।

৬টা Key Point:

1. RDB: interval snapshot — দ্রুত restart, কিন্তু শেষ snapshot-এর পরের ডেটা হারায়; AOF: প্রতিটা write log — কম হারায়, ফাইল বড়
2. প্র্যাকটিক্যাল: দুটোই on (AOF everysec) — সর্বোচ্চ ~১ সেকেন্ড হারানোর ঝুঁকি; pure cache হলে persistence বন্ধও রাখা যায়
3. Replication async — primary মরলে replica-তে failover-এ শেষ কিছু write হারাতে পারে; এই সীমাবদ্ধতা স্বীকার করা
4. Sentinel: monitoring + automatic failover (ছোট/মাঝারি সেটআপ); Cluster: hash slot-এ data ভাগ + HA (বড় স্কেল)
5. Cluster-এ multi-key operation সীমিত — ভিন্ন slot-এর key এক command-এ নয়; hash tag `{user:1}`-এ জোর করা যায়
6. Distributed lock: `SET key val NX EX ttl` — TTL ছাড়া lock নিলে crash-এ চিরতরে আটকে যায়; সুস্থ ক্ষেত্রে Redlock বিতর্কও এক লাইনে জানা ভালো

৮. Queue Worker Scaling ও Backpressure

Description: Queue জমে যাচ্ছে — worker বাড়াবেন নাকি অন্য কিছু? প্রোডাকশনে queue অপারেট করার বাস্তব প্রশ্ন।

৬টা Key Point:

1. প্রথম মেট্রিক queue depth + processing rate — জমার হার > খরচের হার হলেই ব্যবস্থা; monitoring ছাড়া queue চালানো অন্ধ গুড়া
2. Worker বাড়ানো কাজ করে যতক্ষণ bottleneck CPU/worker — bottleneck DB হলে worker বাড়ালে DB-ই আগে মরবে
3. Queue আলাদা করা priority অনুযায়ী: critical (payment/OTP) আলাদা queue + dedicated worker — bulk import-এর পেছনে OTP আটকে থাকা চলবে না

4. Slow job ভাঙা: এক বিশাল job নয়, ছোট ছোট chunk job (batch) — parallel হয়, fail হলে অল্প হারায়
5. Backpressure: producer-কে ধীর করা (rate limit, bounded queue) — সীমাহীন জমতে দিলে memory/latency বিস্ফোরণ
6. Autoscaling সংকেত queue depth দিয়ে (Horizon auto-balance, K8s KEDA) — সাথে max cap, নাহলে খরচ/DB connection ফেটে যায়

৯. Pub/Sub vs Queue — Fan-out ও Event Broadcasting

Description: "একটা ঘটনা অনেকে শুনবে" বনাম "একটা কাজ একজন করবে" — এই পার্থক্য আর real-time notification-এ কীভাবে লাগে।

৬টা Key Point:

1. Queue: প্রতিটা message এক consumer process করে (work distribution); Pub/Sub: সব subscriber কপি পায় (notification)
2. Redis Pub/Sub fire-and-forget — subscriber offline থাকলে message চিরতরে হারায়; তাই শুধু ephemeral কাজে (live update)
3. Redis Streams মাঝামাঝি: persistent log + consumer group — Redis-এই থেকে Kafka-lite দরকার হলে
4. Laravel broadcasting চেইন: event → queue → Reverb/Pusher (WebSocket) → frontend
`Echo.private('orders.5')` — চেইনটা বলতে পারা জরুরি
5. WebSocket-এ authorization ভোলা যাবে না: private channel-এ auth callback — নাহলে যে কেউ অন্যের notification শোনে
6. Fan-out durable লাগলে: RabbitMQ fanout exchange (প্রত্যেক subscriber-এর নিজের queue) বা Kafka consumer group — offline থাকলেও ফিরে এসে পায়

১০. Event-driven Architecture-এর ঝুঁকি ও Observability

Description: সব async/event করলে নতুন সমস্যা আসে — debugging, ordering, consistency। সিনিয়র হিসেবে glamour-এর পাশাপাশি খরচটা জানা যাচাই হয়।

৬টা Key Point:

1. সবচেয়ে বড় খরচ traceability: এক request-এর কাজ ৫ সার্ভিস/queue-তে ছড়ায় — correlation ID প্রতিটা message-এ বহন করা বাধ্যতামূলক
2. Eventual consistency UI-তে দেখা দেয়: order দিলেন, লিস্টে নেই এখনো — UX-এ optimistic update/স্পষ্ট pending state দিয়ে সামলানো
3. Schema evolution: event-এর structure বদলালে পুরনো consumer ভাঙে — versioned event/backward-compatible ফিল্ড যোগ (শুধু add, remove নয়)
4. Monitoring অপরিহার্য চারটা: queue depth, consumer lag (Kafka), DLQ size, processing latency — এগুলোর alert না থাকলে silent failure
5. Duplicate + out-of-order দুটোই স্বাভাবিক ধরে ডিজাইন — idempotency + event-এ timestamp/version দিয়ে stale ignore
6. সংস্কৃতি-নিয়ম: monolith + queue-ই বেশিরভাগ BD-স্কেল প্রোডাক্টে যথেষ্ট — Kafka-ভিত্তিক full event-driven তখনই, যখন একাধিক সিস্টেম সত্যিই একই stream খায়